Lexical Task Heads beyond Simple Tasks: Logit Lens and Jacobian Lens Readouts

Filippo Merlo

Draft — October 29, 2026

Abstract

Lexical task heads are attention heads whose output, decoded into the vocabulary, names the task the prompt asks for (Yang et al., 2026). They were identified with the per-head logit lens on simple single-step tasks, such as mapping a country to its capital. We ask whether the same method finds them for complex tasks defined by explicit rules (Fu et al., 2026), which need several inference steps, and whether Head-JLens (Merlo, 2026), which decodes each head's output after mapping it through a Jacobian of the remaining layers averaged over many texts, finds lexical task heads that the logit lens misses. In Llama-3.1-8B-Instruct, we identify lexical task heads under both readouts on 14 simple tasks and 5 complex tasks at their easy level, with rule-based and example-based prompts, and test each head set by activation patching.

1 Introduction

A language model's accuracy on a task can change a great deal with the wording of the prompt that asks for it: on Llama-3.1-8B-Instruct, prompts for the same antonym items differ in accuracy by up to a factor of four (Yang et al., 2026, Fig. 1). Yang et al. (2026) explain part of this variation with *lexical task heads*, attention heads whose outputs, decoded into the vocabulary with the logit lens, name the task ("opposite" for antonyms, "cities" for capitals). The heads are largely shared between instruction-based and example-based prompts, patching their mean activation into a prompt without task information raises task accuracy, and prompts that fail activate them less.

Two limits of that work motivate this project. First, its 17 tasks are simple single-step mappings (country to capital, word to antonym), and the authors note that "it remains unclear if these shared lexical mechanisms hold for more abstract tasks or tasks that are less easily defined by simple vocabulary clusters" (Yang et al., 2026, Sec. 4). Fu et al. (2026) provide such tasks: five novel tasks defined by explicit rules (a card game, a board game, a composed arithmetic operator, noun-class agreement in an artificial language, and lexical-category inference), each at three difficulty levels and each given as rules, as examples or as both. Second, the logit lens decodes a head's output directly and leaves out the computation of the layers that follow it; Yang et al. (2026, Sec. 4) note that task representations "encoded in a non-lexical, abstract format" may not project cleanly to tokens. Head-JLens reads each head through a Jacobian map of the remaining layers, and on Llama-3.1-8B-Instruct it ranks annotated intermediate words, which the prompt never states (such as the bridge entity of a two-hop question), higher than the per-head logit lens at most layers on five of six evaluation sets (Merlo, 2026, Sec. 3).

We therefore ask whether Yang et al.'s method identifies lexical task heads for the complex tasks of Fu et al. (2026), and how the logit lens and Head-JLens compare as readouts for identifying them.

Contributions.

1. A test of the lexical-task-head method on complex, rule-defined tasks, with a causal check of the identified heads.

2. A comparison of the per-head logit lens and Head-JLens for identifying lexical task heads, on simple and complex tasks.

2 Related work

Task representations across prompting styles. Function vectors are sums of the mean outputs of a few attention heads that, added to the residual stream, trigger a task in another context (Todd et al., 2024). Davidson et al. (2025) extract them from instructions as well as from demonstrations and find that the two prompting styles engage mostly different heads: of the 20 top heads in each model, 2 to 7 are shared. Yang et al. (2026) identify a different kind of task representation, lexical task heads, whose outputs decode to words describing the task; 73% of them are shared between instruction-based and example-based prompts on average, and they are largely disjoint from function-vector heads (Yang et al., 2026, App. I). We test the lexical-task-head method on complex tasks.

Reading attention heads in the vocabulary. The logit lens decodes an intermediate state with the model's final normalization and unembedding (nostalgebraist, 2020); applied to one head's output, it is the readout of Yang et al. (2026). The Jacobian lens of Gurnee et al. (2026) instead maps a residual-stream state through a Jacobian of the remaining layers, averaged over many texts. Head-JLens moves this map to the concatenated head outputs, so that each head is read through the layers that follow it; in Llama-3.1-8B-Instruct it ranks annotated intermediate words higher than the per-head logit lens at most layers on five of six evaluation sets, and heads selected with it are better sites for edits (Merlo, 2026). We compare the two readouts on a different use: identifying task-descriptive heads.

Learning tasks from rules and from examples. Fu et al. (2026) compare learning from rules, from examples and from both on five novel tasks. Rules outperform examples overall, adding examples to rules gives no significant gain, and the advantage of rules is largest for tasks that need algebraic computation. Their study is behavioural; we use their tasks to study task representations.

3 Research questions

RQ1 Does the method of Yang et al. (2026) identify lexical task heads for the complex tasks of Fu et al. (2026) under rule-based and example-based prompts, and do the identified heads have a causal effect on task accuracy?

RQ2 How do the per-head logit lens and Head-JLens compare as readouts for identifying lexical task heads, on simple and on complex tasks?

RQ3 Are the lexical task heads of a complex task shared between rule-based and example-based prompts, as Yang et al. (2026) find for simple tasks?

Hypothesis 1 (replication)**.** *On the simple tasks, Logit-head reproduces Yang et al. (2026): lexical task heads are found for every task, and patching them into prompts without task information raises accuracy above the unpatched baseline.* Falsified *if, on the same model, the method finds no heads or no accuracy gain for tasks where Yang et al. (2026) report both.*

Hypothesis 2 (complex tasks)**.** *Following the limitation stated by Yang et al. (2026, Sec. 4), the complex tasks yield fewer lexical task heads per task and a smaller causal effect than the simple tasks under Logit-head.* Falsified *if they yield as many heads with as large an effect.*

Hypothesis 3 (readouts)**.** *Following Merlo (2026), J-head identifies at least the lexical task heads that Logit-head identifies, and the heads found only by J-head have a causal effect above baseline.* Falsified *if the J-head set misses heads of the Logit-head set, or if the heads found only by J-head have no causal effect.*

Scope. We do not explain behavioural variation across wordings (the failure analyses of Yang et al., 2026, Sec. 2.4), and we use one model.

4 Framework

Tasks and prompts. A task t is given by a dataset D_t of inputs and correct outputs. For the complex tasks we use the easy level of each (Sec. 5.2). Each prompt for t has a style $s \in \{\text{rule}, \text{example}, \text{combined}\}$: its task information is a rule text, a set of demonstrations, or both, followed by the input x. $P_{t,s}$ is the set of prompts of style s for t, and $P^{+}_{t,s} \subseteq P_{t,s}$ those the model answers correctly and straight away, with the answer as the first generated text (in reasoning mode, Sec. 5.3: correctly after reasoning).

Heads and readouts. Following Merlo (2026, Sec. 2), $z_{l,h}(p)$ is the output of head h in layer l at the last position of p, before the output projection, and $c_{l,h}(p) = W^{(h)}_{O,l} z_{l,h}(p)$ its write to the residual stream. With $q_v = g \odot w_v$ the gain-scaled unembedding row of token v, the two readouts score v by

$$\ell^{\mathrm{L}}_{l,h}(p)[v] = q_v^{\top} c_{l,h}(p) \quad \text{(Logit-head)}, \qquad \ell^{\mathrm{J}}_{l,h}(p)[v] = q_v^{\top} J_{l,h}\, z_{l,h}(p) \quad \text{(J-head)}.$$

$T^{R}_{k,l,h}(p)$ is the set of the k highest-scoring tokens under readout $R \in \{\mathrm{L}, \mathrm{J}\}$, and V_t the term list of task t.

Definition 1 (Lexical task head)**.** *Following Yang et al. (2026, Sec. 2.2.2), head (l, h) is a lexical task head for t on prompt p under readout R if $|T^{R}_{k,l,h}(p) \cap V_t| \geq n$. It is a lexical task head for t and style s if this holds for at least a fraction ρ of the prompts in $P^{+}_{t,s}$. We use $n = 1$, $k = 10$, $\rho = 0.1$, as Yang et al. (2026). $H^{R}_{t,s}$ denotes the set of such heads.*

5 Methods

5.1 Model

Llama-3.1-8B-Instruct: it is the main model of Yang et al. (2026), and the only model with released Head-JLens matrices.

5.2 Tasks

Simple tasks. The 14 tasks used by both Yang et al. (2026) and Davidson et al. (2025), with the datasets of Todd et al. (2024): antonym, synonym, singular–plural, present–past, country–capital, country–currency, landmark–country, park–country, product–company, next-item, prev-item, English–French, English–German, English–Spanish. Rule-based prompts: the five instruction templates per task of Yang et al. (2026, App. A.3.1). Example-based prompts: 1, 5, 10, 20 and 30 shots (Yang et al., 2026, App. A.3).

Complex tasks. The five tasks of Fu et al. (2026) at their easy level: Set Game, Tapatan, Operator Function, Noun Class Agreement, Lexical Category Inference. Data from their public release: 300 test items per task, drawn with their sampler and seed (all 600 for Lexical Category Inference), the same items under every prompt. Rule-based prompts: their rules-only prompts (Fu et al., 2026, App. F) and further wordings (Sec. 5.3). Example-based and combined prompts: their minimum-coverage demonstration sets (Fu et al., 2026, Sec. 2.5).

Why these tasks. Lexical task heads and their term lists were defined on the simple tasks (Yang et al., 2026), so the method applies unchanged and our results can be compared with theirs. Yang et al. (2026, Sec. 4) restrict their study to tasks with simple lexical descriptions. The complex tasks are novel, need several inference steps, and are given in rule, example and combined forms that specify the same task on the same test items, so the task is held fixed while the way it is communicated varies (Fu et al., 2026, Sec. 1–2).

Answer format. As in the source studies: the templates of Yang et al. (2026) for the simple tasks; the answer formats and generation limits of Fu et al. (2026, App. A) for the complex tasks, with greedy decoding (Fu et al. sample at temperature 0.1), so that the prompt whose heads are read and the scored answer always match. This keeps accuracy comparable with the reported results. Fu et al.'s instruction-tuned models often explain before answering, so forcing a direct answer would be a different setting. Lexical task heads are read at the last prompt token, where the answer is produced only if the model answers straight away; prompts on which it explains first are therefore left out of $P^+_{t,s}$ (Sec. 4), except for the tasks run in reasoning mode (Sec. 5.3).

5.3 Task-representation methods

Lexical task heads (Yang et al., 2026, Sec. 2.2.2). Each attention head's output at the last prompt token is decoded with the two readouts of Sec. 4: Logit-head, the per-head logit lens used by Yang et al. (2026) (nostalgebraist, 2020), and J-head (Head-JLens), which applies the head's columns $J_{l,h}$ of the per-layer Jacobian map fitted on WikiText-103 (Merlo, 2026, Sec. 2). Fitted matrices for Llama-3.1-8B-Instruct are released with Merlo (2026, Sec. S4). Under each readout, lexical task heads are identified with Definition 1. For the simple tasks, $P^+_{t,s}$ is taken as in the code of Yang et al. (2026): the first 100 prompts answered correctly under one wording, the 5-shot prompts for the example style and the instruction template with the highest accuracy for the rule style. The complex tasks follow the same rule: the first 100 such prompts under the most accurate rule wording, under the example-based prompts and under the combined prompts. Term lists: Yang et al. (2026, App. A.2.1) for the simple tasks; for the complex tasks, see below.

Simple tasks as in the source code. The simple tasks are run as in the code of Yang et al. (2026), with three departures. First, padding: their code passes prompts to the model in batches of 20, padded on the left, without an attention mask, so that the model attends to the padding; we pass the mask and the matching positions, so that each prompt is computed as if run alone. Second, the J-head readout. Third, the precision of the forward pass: bfloat16, in which the Head-JLens matrices were fitted.

What we add to the complex tasks. Fu et al. (2026) provide the data, one rules-only prompt per task and difficulty level, and the example-based and combined prompts. To apply lexical task heads to them we add three things.

1. *Rule wordings*: six per original rule prompt (App. A), each stating exactly the same rules. They are generated with GPT-5.5 through the Azure OpenAI API from a paraphrase

prompt that keeps placeholders, output labels and the answer-format instruction verbatim (App. B), and each is checked by us against the original; App. C lists them. Yang et al. (2026) use five instruction templates per simple task; the complex tasks have six rule wordings besides the original, and their rule-style P^+ is taken from the most accurate of the seven.

2. *Term lists* of task-descriptive words, of the kind in Yang et al. (2026, App. A.2.1): single words that describe the task rather than its answers, with case and number variants. They are generated with GPT-5.5 through the Azure OpenAI API from the rule prompt (App. B), as Yang et al. (2026, App. O) generated term lists with LLMs, and checked by us. Each word is expanded to token ids as in Yang et al. (2026): as written, with a leading space, in upper, capitalised and lower case, and after a `.`, `_` or `-`, keeping every variant that is a single token of the model's tokenizer. There is one list per original rule prompt (App. A): Tapatan's levels share one prompt and change only board size, piece count and line length, whereas the other tasks add rules or, for Operator Function, change the function (Fu et al., 2026, Table 1, App. F). Lists are task-specific, not item-specific: for Lexical Category Inference, whose category definitions change from item to item, the list contains only words that describe the task itself. App. D lists them.

3. *Prompts without task information* for the causal check (Sec. 5.4): the input block of the rules-only prompt alone (the lines that introduce the input and the answer, such as `Sentence: {sentence}` and `Answer:`), without rules or examples, the counterpart of the zero-shot prompts of Yang et al. (2026, Sec. 2.3.2).

Reasoning mode. For the complex tasks on which E0 finds that the model does not answer straight away (Set Game, Operator Function and Tapatan), we follow the instruction-tuned setup of Fu et al. (2026, App. A) and let the model reason: chat template, greedy decoding, and an output limit of 6,000 new tokens, their limit for Operator Function, for all of these tasks. Answers are extracted as in their released code: after whitespace is collapsed, the first `\boxed{}`, read as three cards of the form `(animal | biome | ...)` for Set Game and as an integer for Operator Function. Their code has no parser for Tapatan; we take the first of `A win`, `B win`, `continue` in the output. An output without an extractable answer within the limit is wrong. Generation stops once this answer is complete, since nothing after it can change the score. $P^+_{t,s}$ then contains the prompts whose final answer is correct, whatever precedes it. Definition 1 is applied at two positions: the last prompt token, as for the other tasks, where heads show the task as identified from the prompt; and the pre-answer position, the token after which the first answer token is written, found by one forward pass over the prompt and the model's own output, where heads show the task as represented when the answer is produced. At the pre-answer position, the copy share also checks the generated text. Heads at the last prompt token are compared across all tasks; heads at the pre-answer position and the accuracies after reasoning are reported separately. These tasks use the same 300 items as the others; where items share a prompt (Tapatan's sample repeats 46 boards), the prompt is answered once and its reply scored for each of them.

Why these readouts. Lexical task heads locate a task representation and give its content in words (Yang et al., 2026). Logit-head reproduces their readout. J-head reads each head through the layers that follow it: in Llama-3.1-8B-Instruct it ranks unstated intermediate words higher than Logit-head at most layers on five of six evaluation sets, and its readouts are better guides for edits (Merlo, 2026, Secs. 3–4). A lexical task head's decoded task word is likewise not the answer, and Yang et al. (2026, Sec. 4) note that their logit-lens criterion may miss task representations that do not project cleanly to tokens.

5.4 Causal check

The causal check of Yang et al. (2026, Sec. 2.3.2, App. G) tests whether a head set carries the task: the heads' mean output on task prompts is written into prompts that do not state the task, and we measure whether the model then performs it.

1. *Patched prompts*: prompts without task information, the zero-shot prompts of Yang et al. (2026) for the simple tasks and the input blocks of Sec. 5.3 for the complex tasks, for items that the model answers correctly when the task is given. Their unpatched accuracy is the baseline.

2. *Mean output*: for each head in the set, the mean of $c_{l,h}$ over the first 100 prompts of a wording answered correctly, leaving out the patched items. As in the code of Yang et al. (2026), each wording of a style (each instruction template, each number of shots, each rule wording) gives its own mean and its own patch.

3. *Patch*: at the last position of each patched prompt, the output of every head in the set is replaced by its mean output times a scaling factor $\alpha \in \{-10, -6, -4, -2, 0, 1, 2, \ldots, 10\}$, the values of Yang et al. (2026). The model answers from the patched prompt, and the answer is scored as in E0.

4. *Control*: the same patch at the same heads, with the mean output of a different task. A head set carries the task if its own mean raises accuracy and the control mean does not; a gain under both points to the answer format rather than the task.

Three head sets are patched for each task and style: $H^{\mathrm{L}}_{t,s}$, $H^{\mathrm{J}}_{t,s}$, and the heads found only by J-head, $H^{\mathrm{J}}_{t,s} \setminus H^{\mathrm{L}}_{t,s}$. We report the accuracy at each α averaged over the wordings of the mean's style. In reasoning mode (Sec. 5.3), each set's mean output is taken at the position where the set was found. It is patched at the last prompt position and kept at every generated position, and the final answer is scored. Which items are patched, which means and control tasks are used, and the values of α in reasoning mode are given for each task family in E1 and E2 (Secs. 6.2 and 6.3).

Second check. The code of Yang et al. (2026) releases one script for this check (`causal_fix_incorrect.py`), and it differs from the procedure described in their App. G.1, which we follow above, in the patched prompts. Instead of prompts without task information, it patches prompts that give the task and are answered wrongly: the 1-shot prompts of the items answered wrongly with one shot. These prompts show the answer format, and their unpatched accuracy is 0 by construction. Head sets, means, α and scoring are as above, and the script has no control. We run it as a second check and report both. On a task with two labels, a patch that only moves the model's answers towards one label already raises accuracy on these items, to the share of that label among them; we compare the gains with this share.

5.5 Measures

For each task t, style s and readout R:

- *Accuracy* of each prompt, with the evaluation of the source paper: that of Yang et al. (2026) for the simple tasks and that of Fu et al. (2026) for the complex tasks.

- *Head count* $|H^R_{t,s}|$ and the distribution of the heads over layers.

- *Overlap* between two head sets A and B: $|A \cap B|/|A|$ and $|A \cap B|/|B|$. Compared pairs: $H^{\mathrm{L}}_{t,s}$ and $H^{\mathrm{J}}_{t,s}$ (RQ2); $H^R_{t,\mathrm{rule}}$ and $H^R_{t,\mathrm{example}}$ (RQ3).

- *Causal effect*: the highest accuracy over α minus the unpatched accuracy, for the head set and for the control.
- *Copy share*: the share of prompts on which a head's matched terms ($T^R_{k,l,h}(p) \cap V_t$) occur in the prompt text, averaged over the heads of $H^R_{t,s}$. It separates heads that may copy task words from the prompt from heads that do not.

6 Experiments

Every experiment is run once, with greedy decoding, on Llama-3.1-8B-Instruct. Each one is described below by the conditions it crosses and the steps it runs; the methods of Sec. 5 are referred to where they are defined.

6.1 E0: Behavioural baseline

Question. On which tasks and prompt styles does the model perform above chance, and does it answer straight away?

Conditions.

- *Simple tasks*: the 14 tasks, each on every item of its dataset, under eleven prompts: the five instruction templates (rule style), 1, 5, 10, 20 and 30 shots (example style), and the zero-shot prompt (no task information).
- *Complex tasks*: the five tasks at the easy level, each on the same 300 items (600 for Lexical Category Inference), under ten prompts: the original rules prompt and its six wordings (rule style), the example-based prompt, the combined prompt, and the input block alone (no task information, Sec. 5.3).
- *Mode* (complex tasks): direct, with the answer formats and generation limits of Fu et al. (2026, App. A); and, for the tasks that the direct mode finds are never answered straight away, reasoning mode (Sec. 5.3).

Procedure.

1. Generate the model's answer to every prompt and score it with the evaluation of the source paper: for the simple tasks, the first generated token against the first token of the correct answer; for the complex tasks, the answer parser of Fu et al. (2026).
2. For the complex tasks, record whether the answer is the first generated text (answered straight away). In reasoning mode, score the final answer after reasoning.
3. Compare each accuracy with chance: the share of the most frequent label for the tasks with a fixed label set, 0 for open answers.

Output. Accuracy per task and prompt (Tables 1 and 2); the tasks run in reasoning mode; for each task and style, the prompts $P^+_{t,s}$ used by E1 and E2 and, for the rule style, the most accurate wording from which they are taken; the items answered correctly and wrongly, which select the patched items of the causal checks.

6.2 E1: Simple tasks

Question. H1, and RQ2 on the simple tasks: does Logit-head reproduce Yang et al. (2026) on our setup, and how does J-head differ from it where the method is known to work?

Conditions. 14 tasks × 2 styles (rule, example) × 2 readouts (Logit-head, J-head). Each pair of task and style gives three head sets: $H^{\mathrm{L}}_{t,s}$, $H^{\mathrm{J}}_{t,s}$, and the heads found only by J-head, $H^{\mathrm{J}}_{t,s} \setminus H^{\mathrm{L}}_{t,s}$.

Procedure.

1. *Identification prompts*: $P^{+}_{t,\mathrm{rule}}$, the first 100 prompts answered correctly under the most accurate of the five instruction templates; $P^{+}_{t,\mathrm{example}}$, the first 100 answered correctly with five shots.

2. *Head sets*: decode every head at the last prompt token under both readouts and apply Definition 1.

3. *Measures* (Sec. 5.5): head count and layers of each set; overlap of the Logit-head and J-head sets of each style (RQ2), and of the rule-style and example-style sets of each readout; copy share.

4. *First causal check* (Sec. 5.4). Patched items: the first 100 items answered correctly with one shot. Patched prompts: their zero-shot prompts, whose unpatched accuracy is the baseline. Means: one per source wording, each over the first 100 prompts of that wording answered correctly, leaving out the patched items; there are ten, the five instruction templates and the five shot counts. Each head set is patched with the five means of its own style and, across styles, with the five means of the other style. Control: the mean of the control task of Yang et al. (2026) at the same heads, one per style of mean, taken as in their code: over the control task's 1-shot prompts, at all the items it answers correctly with one shot (for example-style means) or under the second instruction template (for rule-style means). Scaling: the 15 values of α in Sec. 5.4; $\alpha = 1$ writes the mean itself, $\alpha = 0$ sets the heads to zero, and a negative α reverses its sign.

5. *Second causal check* (Sec. 5.4). Patched items: the first 100 items answered wrongly with one shot, or all of them where fewer are wrong. Patched prompts: their 1-shot prompts, whose unpatched accuracy is 0 by construction. Head sets, means and α as in the first check; no control.

6. *Scoring*: the first token generated from each patched prompt, as in E0. Accuracy at each α is averaged over the five means of a style; the causal effect is the highest of these averages minus the baseline.

Output. Head sets and measures (Table 3, Figures 1 and 2); causal effects with means of the same style (Tables 4 and 6) and of the other style (Tables 5 and 7); accuracy against α (Figures 3 and 4).

6.3 E2: Complex tasks

Question. RQ1, H2, RQ2 on the complex tasks, and RQ3: are lexical task heads found and causally effective for complex tasks, under which readout, and are they shared between rule-based and example-based prompts?

Conditions. 5 tasks × 3 styles (rule, example, combined) × 2 readouts, with the same three head sets per pair of task and style as in E1. Noun Class Agreement and Lexical Category Inference are run in direct mode; Set Game, Operator Function and Tapatan in reasoning mode, where each pair is read at two positions: the last prompt token and the pre-answer position (Sec. 5.3).

Procedure.

1. *Identification prompts*: for each style, the first 100 prompts in $P^{+}_{t,s}$ (answered correctly and straight away, or in reasoning mode correctly after reasoning): under the most accurate of the seven rule wordings, under the example-based prompt and under the combined prompt. Where fewer prompts qualify, all of them are used.

2. *Head sets*: Definition 1 with the term list of the task (Sec. 5.3), at each position.

3. *Measures*: as in E1, with the overlap of the rule-style and example-style sets for RQ3; at the pre-answer position, the copy share also checks the generated text.

4. *First causal check*, rule-style head sets only. Patched items: the first 100 items in P^{+} under the original rules prompt (the first 10 in reasoning mode). Patched prompts: their prompts without task information (the input blocks of Sec. 5.3). Means: one per rule wording, seven in all, each over the first 100 prompts of that wording in P^{+}, leaving out the patched items. Control: the mean of a task with the same answer format under its original rules prompt (Noun Class Agreement and Lexical Category Inference with each other, Set Game and Operator Function with each other, Tapatan with Set Game). Scaling: the 15 values of α, as in E1.

5. *Second causal check*, rule-style head sets only. Patched items: the first 100 items answered wrongly under the original rules prompt (the first 10 in reasoning mode). Patched prompts: their original rules prompts, whose unpatched accuracy is 0 by construction. Means and α as in the first check; no control. On the two-label tasks, the accuracy of answering every patched item with one label, the share of that label among the patched items, is the reference that a shift towards one answer reaches without any task information.

6. *Reasoning mode*: in both checks, $\alpha \in \{0, 1, 4, 10\}$, to bound the cost of long generations. Each set's mean is taken at the position where the set was found; the patch is applied at the last prompt token and kept at every generated token, and the final answer is scored.

7. *Scoring*: the answer generated from each patched prompt, scored as in E0. Accuracy at each α is averaged over the seven means; the causal effect is computed as in E1.

Output. Head sets and measures (Table 8, Figures 1 and 2); causal effects (Tables 9 and 10); accuracy against α (Figure 5).

7 Results

Tables 1–10 and Figures 1–5 are generated from the result files by `scripts/results_tex.py`. Causal-check accuracies are over the patched items (Sec. 6); in reasoning mode one item is 0.1 (0.14 in Operator Function's second check, which has 7 items). Still running: the causal checks of Tapatan.

7.1 E0: Behavioural baseline

Simple tasks (Table 1). Every task is answered well above its zero-shot accuracy under both styles, and the zero-shot prompts are almost never answered correctly. Accuracy rises from one to five shots and changes little after that. The instruction templates differ most on the translation tasks, next-item, prev-item, park–country and synonym, where one or two templates fall well below the others.

		Rule (5 templates)		Example (shots)					
Task	Items	mean	range	1	5	10	20	30	Zero-shot
antonym	2398	0.62	0.53–0.65	0.32	0.66	0.69	0.69	0.69	0.00
synonym	2880	0.44	0.29–0.51	0.26	0.44	0.47	0.51	0.52	0.00
singular–plural	205	0.98	0.96–0.99	0.89	0.98	0.99	0.99	0.99	0.00
present–past	293	0.95	0.84–0.99	0.79	0.98	0.98	0.99	0.99	0.00
country–capital	197	0.98	0.96–0.98	0.87	0.98	0.97	0.97	0.97	0.00
country–currency	197	0.74	0.68–0.82	0.69	0.74	0.79	0.81	0.84	0.01
landmark–country	836	0.84	0.77–0.88	0.77	0.90	0.90	0.91	0.91	0.00
park–country	749	0.79	0.48–0.91	0.94	0.97	0.97	0.99	0.99	0.00
product–company	522	0.73	0.64–0.81	0.70	0.84	0.85	0.87	0.89	0.04
next-item	225	0.42	0.20–0.59	0.56	0.98	0.97	0.98	0.99	0.00
prev-item	225	0.54	0.20–0.74	0.54	0.96	0.96	0.96	0.97	0.00
English–French	4698	0.62	0.19–0.79	0.60	0.81	0.82	0.82	0.83	0.00
English–German	5145	0.41	0.10–0.51	0.39	0.53	0.53	0.56	0.56	0.00
English–Spanish	5199	0.71	0.25–0.84	0.69	0.84	0.84	0.85	0.85	0.00

Table 1: E0, simple tasks: accuracy per prompt style. Rule: the five instruction templates of Yang et al. (2026), mean and range; example: by number of shots; zero-shot: the prompts without task information. Items: dataset size.

			Rule					No task	
Task	Mode	Items	orig.	w1–6	range	Examples	Combined	info	Chance
Noun Class Agreement	direct	300	0.63	0.59	0.55–0.61	0.58	0.35	0.00	0.50
Lexical Category Inference	direct	600	0.67	0.66	0.65–0.69	0.59	0.12	0.00	0.50
Set Game	reasoning	300	0.58	0.57	0.51–0.64	0.00	0.53	0.32	0.00
Operator Function	reasoning	300	0.98	0.56	0.01–0.99	0.00	0.99	0.00	0.00
Tapatan	reasoning	300	0.42	0.40	0.36–0.44	0.30	0.16	0.00	0.34

Table 2: E0, complex tasks at the easy level: accuracy per prompt. Mode: direct, with the answer limits of Fu et al. (2026); reasoning, as in Sec. 5.3, accuracy after reasoning. Rule: the original rules prompt (orig.) and the six added wordings (w1–6: mean, and range). No task info: the prompt without task information. Chance: share of the most frequent label for the tasks with a fixed label set, 0 for open answers.

Complex tasks (Table 2). Noun Class Agreement and Lexical Category Inference are answered straight away and above chance under all seven rule wordings and under examples. Combined prompts fall below chance, largely because the model often explains before answering: it answers straight away on 65% (Noun Class Agreement) and 17% (Lexical Category Inference) of combined prompts, against at least 96% under every rule and example prompt. Without task information it never answers straight away.

Set Game, Operator Function and Tapatan are not answered straight away on any of their 300 items under any prompt, so they are run in reasoning mode (Sec. 5.3); Table 2 gives their accuracy after reasoning. Set Game and Operator Function are then solved under rules and combined prompts, and from examples alone on one item of 300 each, so their example-style P^+ is a single prompt. Operator Function varies most across wordings: every wrong answer under its rule wordings is a reply without a `\boxed{}` answer, which the scoring counts as wrong, and under wording 2 only 3 of 300 replies have one. Tapatan stays close to chance under all seven rule wordings and falls below it with examples and combined prompts, so its P^+ sets likely include items answered correctly by chance. Set Game is solved on 96 of 300 items without task information: its input block still asks which three cards on the board form a set.

7.2 E1: Simple tasks

Head sets (Table 3, Figures 1 and 2). Logit-head finds lexical task heads for every task under both styles, from two heads (synonym) upward. J-head finds more heads than Logit-head on 23 of the 28 pairs of task and style. Two thirds of the heads it adds lie in layers 10 to 19, whereas most Logit-head heads lie in layer 20 or later (Figure 1). The J-head set contains the whole Logit-head set on 5 of the 28 pairs, and at least half of it on every pair. Rule-style sets are at least as large as example-style sets on every task, and most heads of an example-style set are also in the rule-style set, except on synonym under both readouts and singular–plural under Logit-head. The copy share is high under most rule-style prompts, whose instructions contain task words, and lower on country–capital, singular–plural and synonym, where the heads also decode task words that the instruction does not contain; under example-style prompts it is close to zero except on country–capital (Figure 2).

Causal check (Tables 4 and 5, Figure 3). Patching the mean output of the Logit-head set, with means of its own style, raises accuracy above the unpatched baseline on country–capital, landmark–country, park–country and product–company under both styles, on country–currency, next-item and singular–plural with examples, and on prev-item with rules. It has no effect on antonym, synonym, present–past and the three translation tasks. Gains appear at positive α. The control mean raises accuracy much less than the task mean, except on park–country, next-item and prev-item, where it raises it by a comparable amount or more.

The J-head set raises accuracy about as much as the Logit-head set or more on most tasks, and also on synonym, where the Logit-head set does not; landmark–country with examples is the clearest case where it raises it less. The heads found only by J-head raise accuracy by themselves on most tasks where the J-head set does, most strongly on singular–plural and country–capital with examples. The control mean at those heads raises it less, except on next-item and prev-item, where it raises it as much or more.

Patched with the means of the other style, the head sets raise accuracy on the same tasks, often by a comparable amount or more. On antonym and singular–plural, the rule-style sets raise accuracy only with example-style means: on antonym the Logit-head set, on singular–plural the J-head set and the heads only it finds. Present–past and the translation tasks show no gain under any pairing.

Second causal check (Tables 6 and 7, Figure 4). Every head set raises accuracy above zero on every task, under both styles and with means of either style, though only by a few items on synonym and the translation tasks. This includes present–past and the translation tasks, where patching the zero-shot prompts has no effect, and antonym, where it has an effect only across styles. The gains are largest on singular–plural, country–capital and present–past and smallest on synonym and the translation tasks, the order reported by Yang et al. (2026, Figs. 57 and 62). The curves also share features of theirs that the zero-shot prompts do not show, such as a gain at negative α on park–country. On landmark–country, park–country and product–company the gains are smaller than with the zero-shot prompts and closer to the reported ones. The J-head set raises accuracy about as much as the Logit-head set or more on most tasks, and far more on next-item and prev-item with examples. The heads found only by J-head raise it on every task where that set is not empty, least on landmark–country with examples.

Hypotheses. For H1, lexical task heads are found for every simple task. With the zero-shot prompts described by Yang et al. (2026, App. G.1), patching them raises accuracy on eight tasks with means of the heads' own style, and on antonym with means of the other style; there is no gain on synonym under Logit-head, on present–past or on the three translation tasks. On three of the eight, park–country, next-item and prev-item, the control mean raises accuracy as well. With the 1-shot prompts of their released script, patching raises accuracy on all fourteen tasks,

	Rule			Example			Rule ∩ example	
Task	L	J	L∩J	L	J	L∩J	L	J
antonym	21	26	18	10	15	9	8	12
synonym	2	22	1	2	12	2	0	6
singular–plural	20	24	14	19	20	12	8	12
present–past	10	7	6	6	4	4	4	3
country–capital	21	37	20	12	36	12	12	25
country–currency	17	33	16	15	28	13	13	22
landmark–country	23	27	19	16	14	10	12	9
park–country	24	24	18	14	12	9	12	9
product–company	30	40	25	16	25	12	12	16
next-item	35	47	29	9	17	6	6	13
prev-item	20	35	19	7	17	5	4	9
English–French	11	21	9	5	13	5	4	10
English–German	11	18	9	6	14	5	6	11
English–Spanish	5	14	5	3	10	3	3	6

Table 3: E1: lexical task heads per task and style under Logit-head (L) and J-head (J). L∩J: heads in both sets. Rule ∩ example: heads in both the rule-style and the example-style set of a readout. Identification prompts: 100 per task and style, the first answered correctly under the most accurate instruction template (rule) and under 5-shot prompts (example).

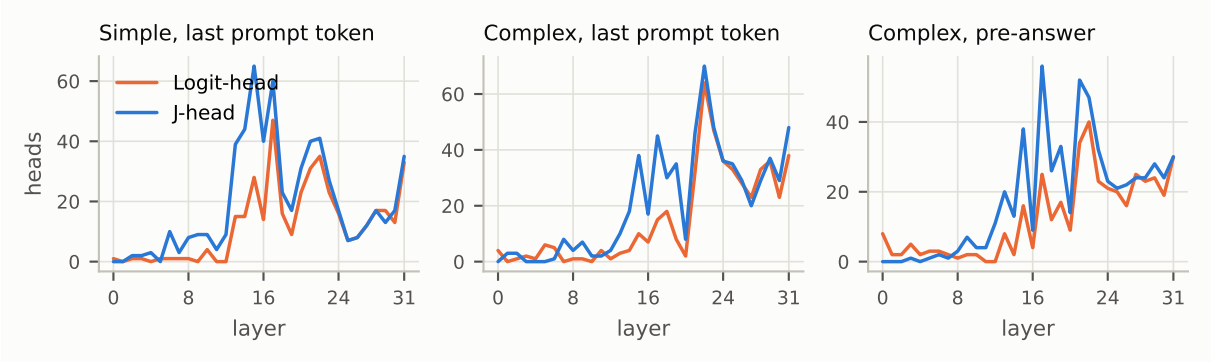


Figure 1: Lexical task heads per layer under each readout, summed over the head sets of all tasks and styles. Middle: Noun Class Agreement, Lexical Category Inference and the reasoning-mode tasks, at the last prompt token. Right: the reasoning-mode tasks at the pre-answer position.

in the per-task pattern of their Figs. 57 and 62. H1 is therefore falsified for the procedure their paper describes and holds for the setting of their released code, which reproduces their figures: the gains they report on present–past and the translation tasks need prompts that show the answer format. For H3 on the simple tasks, the first condition fails, since the J-head set misses part of the Logit-head set on 23 of 28 pairs, and the second holds in both checks: heads found only by J-head have causal effects of their own.

7.3 E2: Complex tasks

Head sets (Table 8). Both readouts find lexical task heads for every complex task and style with a non-empty P^+, at the last prompt token and at the pre-answer position. The sets are larger than for the simple tasks, against H2, but the head counts of the two families are not comparable. The complex term lists expand to 114 to 340 token ids and the simple ones to 2 to 22, and a longer list makes a match among a head's ten top tokens more likely for any head. Small P^+ sets add to this: for the example style of Set Game and Operator Function, whose P^+ is one prompt, every head that matches on it is a lexical task head (9 to 21 heads per set), and for Tapatan's example and combined styles 9 of 89 and 5 of 47 matching prompts suffice. As

			Task mean			Control mean		
Task	Style	Baseline	L	J	J only	L	J	J only
antonym	rule	0.00	0.00	0.00	0.02	0.18	0.00	0.00
	example		0.06	0.03	0.00	0.00	0.00	0.00
synonym	rule	0.01	0.00	0.13	0.13	0.01	0.00	0.01
	example		0.01	0.19	0.27	0.02	0.04	0.05
singular–plural	rule	0.00	0.04	0.02	0.01	0.02	0.10	0.10
	example		0.34	0.85	0.88	0.14	0.18	0.16
present–past	rule	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	example		0.00	0.00	–	0.00	0.00	–
country–capital	rule	0.00	0.69	0.79	0.64	0.01	0.00	0.01
	example		0.73	0.98	0.71	0.02	0.02	0.03
country–currency	rule	0.00	0.09	0.15	0.09	0.04	0.01	0.05
	example		0.18	0.34	0.33	0.00	0.02	0.13
landmark–country	rule	0.00	0.79	0.75	0.34	0.00	0.10	0.12
	example		0.73	0.39	0.01	0.00	0.00	0.04
park–country	rule	0.00	0.36	0.46	0.18	0.28	0.31	0.14
	example		0.58	0.54	0.00	0.29	0.17	0.00
product–company	rule	0.09	0.23	0.34	0.12	0.01	0.00	0.03
	example		0.22	0.27	0.19	0.01	0.06	0.05
next-item	rule	0.01	0.08	0.22	0.31	0.19	0.33	0.48
	example		0.27	0.40	0.44	0.11	0.44	0.65
prev-item	rule	0.01	0.16	0.24	0.23	0.20	0.27	0.08
	example		0.09	0.27	0.29	0.10	0.21	0.16
English–French	rule	0.00	0.00	0.01	0.01	0.00	0.00	0.00
	example		0.00	0.00	0.00	0.00	0.00	0.00
English–German	rule	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	example		0.00	0.00	0.01	0.00	0.00	0.00
English–Spanish	rule	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	example		0.00	0.00	0.00	0.00	0.00	0.00

Table 4: E1 causal check: causal effect, the highest accuracy over $\alpha \in [-10, 10]$ minus the unpatched accuracy (baseline), on the 100 patched items of each task. The head set of each style is patched with means of the same style, and accuracy is averaged over the style's five source wordings before the maximum is taken. Task mean: the head set's mean output over the task's own prompts; control mean: over the control task's prompts (Sec. 6.2). L: Logit-head set; J: J-head set; J only: heads in the J-head set and not in the Logit-head set. –: empty set.

			Task mean			Control mean		
Task	Heads	Baseline	L	J	J only	L	J	J only
antonym	rule	0.00	0.66	0.00	0.00	0.16	0.00	0.00
	example		0.01	0.01	0.01	0.00	0.00	0.00
synonym	rule	0.01	0.00	0.03	0.03	0.01	−0.01	0.01
	example		0.02	0.15	0.17	0.02	0.06	0.07
singular–plural	rule	0.00	0.02	0.70	0.83	0.02	0.11	0.10
	example		0.11	0.05	0.01	0.14	0.18	0.18
present–past	rule	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	example		0.00	0.00	–	0.00	0.00	–
country–capital	rule	0.00	0.73	0.70	0.05	0.01	0.00	0.01
	example		0.78	0.89	0.65	0.02	0.02	0.03
country–currency	rule	0.00	0.15	0.36	0.33	0.04	0.02	0.05
	example		0.10	0.18	0.07	0.00	0.02	0.13
landmark–country	rule	0.00	0.70	0.97	0.95	0.00	0.12	0.13
	example		0.89	0.86	0.01	0.00	0.00	0.04
park–country	rule	0.00	0.33	0.65	0.15	0.27	0.31	0.14
	example		0.50	0.55	0.01	0.29	0.18	0.00
product–company	rule	0.09	0.31	0.36	0.10	0.01	0.00	0.03
	example		0.25	0.31	0.20	0.01	0.05	0.06
next-item	rule	0.01	0.26	0.39	0.49	0.15	0.31	0.45
	example		0.11	0.32	0.38	0.09	0.43	0.50
prev-item	rule	0.01	0.23	0.26	0.11	0.18	0.23	0.07
	example		0.06	0.31	0.31	0.13	0.25	0.21
English–French	rule	0.00	0.00	0.01	0.01	0.00	0.01	0.00
	example		0.00	0.00	0.00	0.00	0.00	0.01
English–German	rule	0.00	0.00	0.01	0.00	0.00	0.00	0.00
	example		0.00	0.00	0.01	0.00	0.00	0.00
English–Spanish	rule	0.00	0.00	0.00	0.00	0.00	0.00	0.00
	example		0.00	0.00	0.00	0.00	0.00	0.00

Table 5: E1 causal check across styles, as Table 4, with the head set of each style (Heads) patched with the means of the other style: rule-style heads with the means of the five shot counts, example-style heads with the means of the five instruction templates.

			Task mean			Control mean		
Task	Style	Baseline	L	J	J only	L	J	J only
antonym	rule	0.00	0.26	0.32	0.23	–	–	–
	example		0.32	0.35	0.24	–	–	–
synonym	rule	0.00	0.04	0.14	0.15	–	–	–
	example		0.04	0.10	0.09	–	–	–
singular–plural	rule	0.00	0.62	0.78	0.52	–	–	–
	example		0.81	0.94	0.90	–	–	–
present–past	rule	0.00	0.43	0.36	0.17	–	–	–
	example		0.61	0.57	–	–	–	–
country–capital	rule	0.00	0.81	0.88	0.70	–	–	–
	example		0.74	0.77	0.70	–	–	–
country–currency	rule	0.00	0.24	0.19	0.22	–	–	–
	example		0.35	0.32	0.26	–	–	–
landmark–country	rule	0.00	0.33	0.33	0.32	–	–	–
	example		0.34	0.31	0.05	–	–	–
park–country	rule	0.00	0.25	0.25	0.21	–	–	–
	example		0.47	0.46	0.27	–	–	–
product–company	rule	0.00	0.15	0.22	0.25	–	–	–
	example		0.17	0.37	0.31	–	–	–
next-item	rule	0.00	0.17	0.35	0.33	–	–	–
	example		0.39	0.80	0.79	–	–	–
prev-item	rule	0.00	0.11	0.09	0.19	–	–	–
	example		0.15	0.76	0.77	–	–	–
English–French	rule	0.00	0.10	0.22	0.22	–	–	–
	example		0.12	0.16	0.15	–	–	–
English–German	rule	0.00	0.06	0.08	0.11	–	–	–
	example		0.09	0.07	0.13	–	–	–
English–Spanish	rule	0.00	0.10	0.23	0.22	–	–	–
	example		0.04	0.12	0.12	–	–	–

Table 6: E1, second causal check, as Table 4, with the 1-shot prompts of the first 100 items answered wrongly with one shot patched; fewer items are wrong on singular–plural (23), country–capital (25), park–country (43), present–past and country–currency (61) and next-item (98). Unpatched accuracy is 0 by construction. No control mean.

			Task mean			Control mean		
Task	Heads	Baseline	L	J	J only	L	J	J only
antonym	rule	0.00	0.40	0.37	0.25	–	–	–
	example		0.25	0.28	0.22	–	–	–
synonym	rule	0.00	0.04	0.09	0.11	–	–	–
	example		0.04	0.20	0.20	–	–	–
singular–plural	rule	0.00	0.70	0.90	0.89	–	–	–
	example		0.75	0.82	0.32	–	–	–
present–past	rule	0.00	0.57	0.40	0.21	–	–	–
	example		0.45	0.45	–	–	–	–
country–capital	rule	0.00	0.71	0.77	0.42	–	–	–
	example		0.84	0.88	0.77	–	–	–
country–currency	rule	0.00	0.34	0.27	0.19	–	–	–
	example		0.27	0.25	0.21	–	–	–
landmark–country	rule	0.00	0.32	0.38	0.38	–	–	–
	example		0.32	0.32	0.08	–	–	–
park–country	rule	0.00	0.31	0.23	0.19	–	–	–
	example		0.35	0.39	0.23	–	–	–
product–company	rule	0.00	0.24	0.31	0.30	–	–	–
	example		0.17	0.31	0.31	–	–	–
next-item	rule	0.00	0.51	0.70	0.70	–	–	–
	example		0.06	0.38	0.38	–	–	–
prev-item	rule	0.00	0.39	0.29	0.16	–	–	–
	example		0.08	0.27	0.32	–	–	–
English–French	rule	0.00	0.11	0.21	0.22	–	–	–
	example		0.10	0.14	0.15	–	–	–
English–German	rule	0.00	0.08	0.07	0.10	–	–	–
	example		0.07	0.07	0.09	–	–	–
English–Spanish	rule	0.00	0.05	0.16	0.15	–	–	–
	example		0.09	0.16	0.15	–	–	–

Table 7: E1, second causal check across styles, as Table 5.

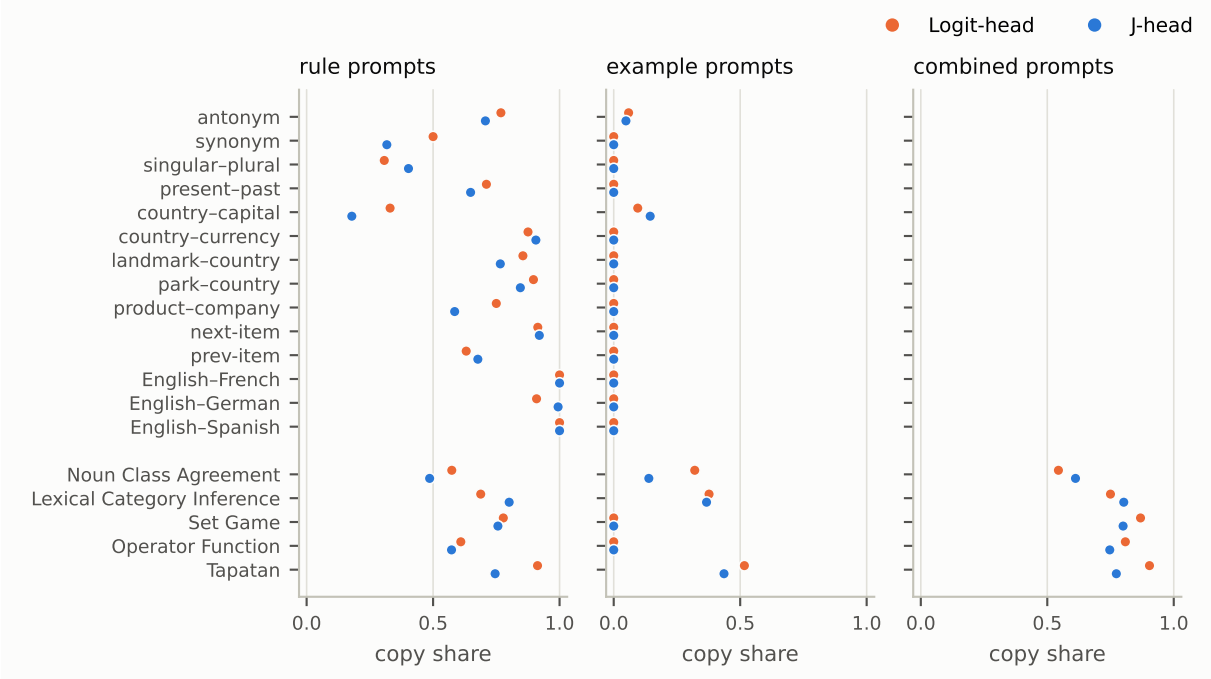


Figure 2: Copy share of each head set (Sec. 5.5): the share of prompts on which a head's matched terms occur in the prompt text, averaged over the set. Complex tasks at the last prompt token; missing points are empty sets.

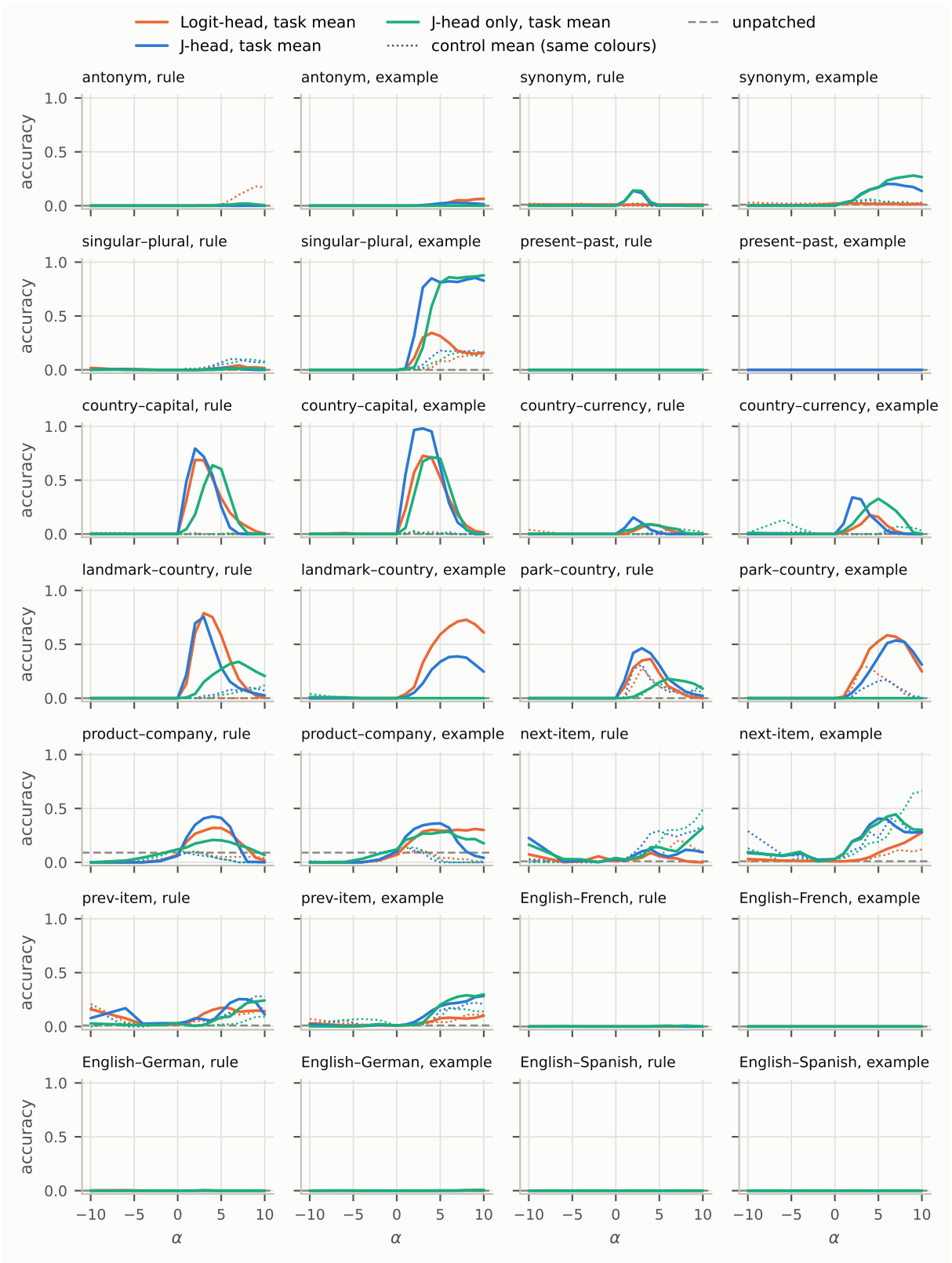


Figure 3: E1 causal check: accuracy on the 100 patched items of each task against α, with each style's head set patched with means of the same style, averaged over the style's five source wordings. Solid: the head set's mean over the task's prompts; dotted: the control mean at the same heads; dashed: unpatched accuracy.

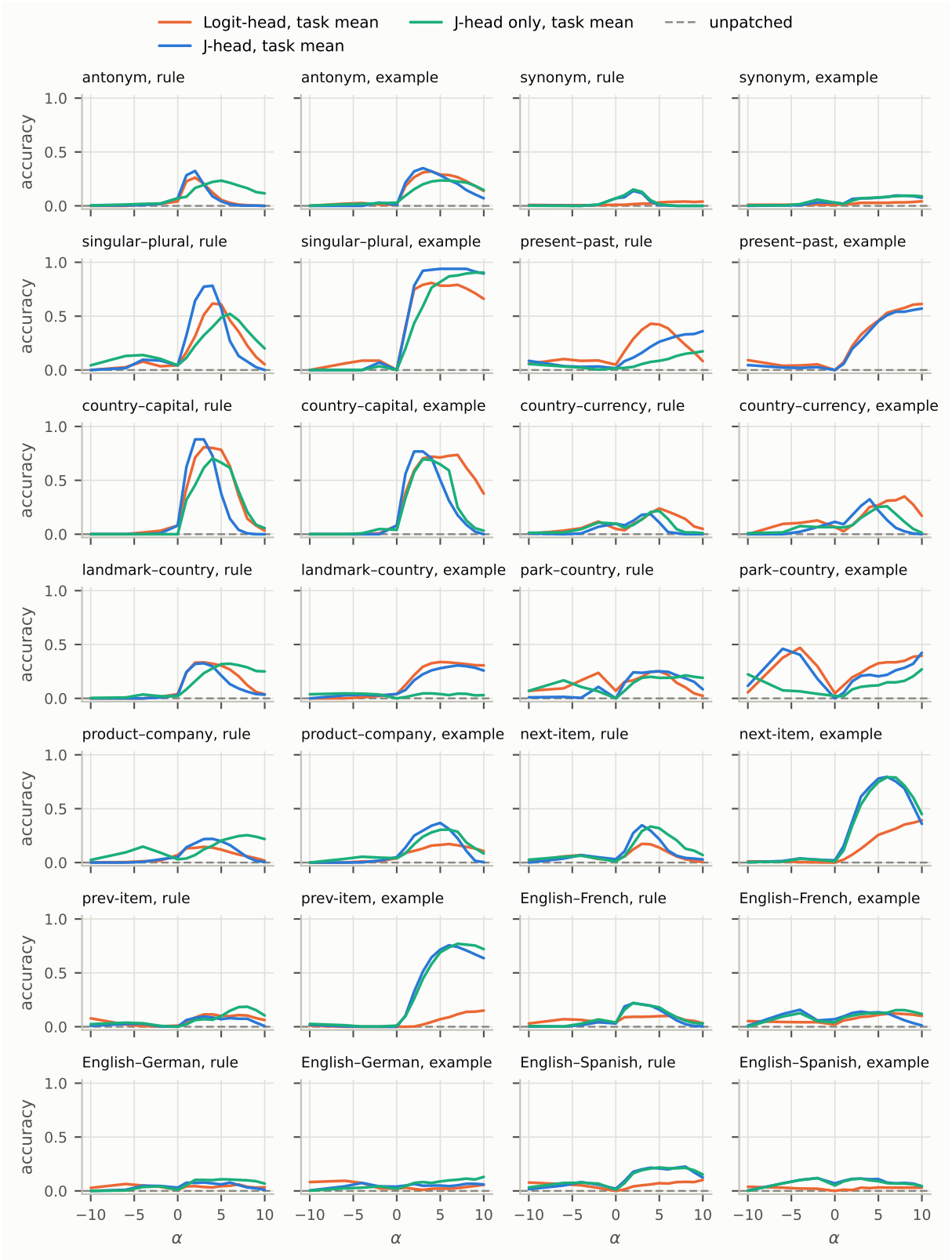


Figure 4: E1, second causal check, as Figure 3, with the wrongly answered 1-shot prompts patched. No control mean.

on the simple tasks, J-head finds more heads than Logit-head on every pair, and three fifths of its added heads lie in layers 10 to 19. Its set covers 44% to 91% of the Logit-head set, never all of it. Of each example-style set, 43% to 93% is also in the rule-style set, more than half on all but two of the 16 pairs of task, position and readout (RQ3), as on most simple tasks; here the example-style sets of Set Game and Operator Function rest on one prompt. At the pre-answer position Tapatan has more heads than at the last prompt token, and Set Game and Operator Function similar numbers under rule and combined prompts. Copy shares lie between about half and nine tenths under rule and combined prompts and are lower under example prompts: zero on Set Game and Operator Function, and well above the near-zero values of the simple tasks' example prompts on the other three tasks (Figure 2).

Causal check (Tables 9 and 10, Figure 5). On Noun Class Agreement and Lexical Category Inference, the prompts without task information get no patched item right, since the model gives no answer straight away. On Noun Class Agreement, no patch changes this, for any head set, mean or α. On Lexical Category Inference, the Logit-head and J-head sets change nothing with their own means. The heads found only by J-head, with the mean of wording 4, make the model answer "No" at α from 5 to 7, to 89 of the 100 items at $\alpha = 6$, which reaches 0.56 (0.08 averaged over the seven wordings); the control mean at the J-head set and at the heads found only by J-head reaches 0.61, the share of "No" among the patched items (the constant-answer line in Figure 5). As in the earlier run with pooled means, the only gains are a constant "No", which the control produces as well: they fit a patch that makes the model give an answer of the required form, not one that supplies the task.

In the second check, every head set raises accuracy above zero on both tasks, but only by moving answers towards one label. At $\alpha = 1$, the sets turn at most 21 of the 100 answers into correct ones under any one wording's mean, and at most 12 averaged over the seven. The largest gains come at α far from 1, where the model gives the same answer to (almost) every item. On Noun Class Agreement, every set answers "No" at $\alpha = -10$ (0.42, the share of "No" among these items), and the Logit-head set answers "Yes" at $\alpha = 9$ (0.58, the share of "Yes"), with each of the seven means. On Lexical Category Inference, setting the Logit-head set to zero ($\alpha = 0$) makes the model answer "Yes" to most items (0.59, against 0.73 for answering "Yes" to all); α from 3 to 6 makes every set answer "No" (0.27); and the heads found only by J-head, with the means of the original prompt and wordings 2 and 4, answer "Yes" to almost every item at α of 7 or 8 (0.69 to 0.73). On these two-label tasks, the second check does not separate task information from a shift towards one answer.

On Set Game in reasoning mode, 3 of the 10 patched items are solved without task information. At the last prompt token, no patch raises accuracy above this: the Logit-head set with its own means reaches at most 4 items with one wording's mean (0.21 averaged over the seven), every other patch stays at or below 3, and in the second check no patch solves more than one item. At the pre-answer position, the three head sets patched at $\alpha = 1$ with their own means solve about 6 of the 10 items (0.56 to 0.60 averaged over the seven means, and at least 4 with each mean), whereas the control means at the same heads and α solve 4 (Logit-head set), 2 (heads found only by J-head) and none (J-head set). Most of the items solved are ones the unpatched prompt gets wrong, and the patch loses some that it gets right: over the 21 conditions (three sets, seven means), the three items solved unpatched stay solved in 26 of 63 cases. Larger α lowers accuracy, to 0 at $\alpha = 10$. In the second check at the pre-answer position, the patches reach at most 0.34 averaged over the seven means (the Logit-head set at $\alpha = 1$), but setting the same heads to zero ($\alpha = 0$) already solves 3 of the 10 wrongly answered items: on items answered after long reasoning, any change to the heads can change the outcome. The pre-answer gain in the first check is the only one on a complex task that the control mean does not match; it rests on 10 items.

On Operator Function, the prompt without task information gives the operands but not the

	Rule				Example				Combined				Rule ∩ ex.	
Task	$\lvert P^+\rvert$	L	J	L∩J	$\lvert P^+\rvert$	L	J	L∩J	$\lvert P^+\rvert$	L	J	L∩J	L	J
Noun Class Agreement	100	18	22	10	100	12	17	6	100	22	35	17	8	10
Lexical Category Inference	100	39	51	31	100	16	22	10	73	46	61	39	13	15
Set Game	100	55	75	48	1	9	13	4	100	52	65	40	6	8
pre-answer	100	54	62	42	1	12	21	6	100	56	72	46	8	9
Operator Function	100	24	41	20	1	9	14	6	100	22	38	20	5	10
pre-answer	100	26	41	17	1	9	14	5	100	19	42	13	4	13
Tapatan	100	61	76	46	89	46	62	38	47	55	70	48	32	40
pre-answer	100	69	94	52	89	70	112	58	47	85	112	67	44	64

Table 8: E2: lexical task heads per task and style. $\lvert P^+\rvert$: identification prompts, the first (at most 100) answered correctly under the most accurate rule wording (rule; original prompt for Noun Class Agreement, wording 2 for Lexical Category Inference, wording 5 for Set Game and Operator Function, wording 1 for Tapatan), under the example-based and under the combined prompts. L, J, L∩J and Rule ∩ ex. as in Table 3. Reasoning-mode tasks: heads at the last prompt token, and (pre-answer) at the pre-answer position.

rule, and on all 10 patched items the model asks for the function instead of answering. No patch supplies it: over all head sets, means and α, at both positions, one patch solves one item. In the second check, only 7 of the 300 items are answered wrongly under the original rules prompt, and every one of these replies reaches the right number but gives it without `\boxed{}` (Sec. 7.1). Changing the Logit-head set makes the model add it: setting the set to zero ($\alpha = 0$) solves 4 of the 7 items at the last prompt token and all 7 at the pre-answer position, and its own means at $\alpha = 1$ solve all 7 at the last prompt token with each of the seven means. The J-head set solves at most one item at the last prompt token and 5 or 6 at the pre-answer position, where zero-ablation already solves 5; the heads found only by J-head solve at most 2, except with the mean of wording 3 at the pre-answer position (all 7). At $\alpha = 10$ no patch solves any item. These gains change the form of the answer, not the answer, as on the two direct-answer tasks.

Hypotheses. For RQ1, the method finds lexical task heads for every complex task under both readouts, but on the two direct-answer tasks neither check shows a task-specific effect for them, nor on Set Game at the last prompt token. At Set Game's pre-answer position, the heads' own means raise accuracy above the unpatched prompt and above the control mean, on 10 items. On Operator Function no patch supplies the missing rule, and the gains of the second check restore the answer format, which zero-ablation does as well; the checks of Tapatan are still running. This largely agrees with the causal part of H2; its head-count part cannot be tested with term lists of such different lengths. For H3 on the complex tasks, the first condition fails, since the J-head set misses part of the Logit-head set on every pair, and so does the second on the direct-answer tasks, at Set Game's last prompt token and on Operator Function, where the heads found only by J-head have no task-specific effect; at Set Game's pre-answer position they have one, on 10 items. Together with E1, H3 is falsified as stated, although on the simple tasks the heads that J-head adds have causal effects of their own.

8 Plan

Steps.

1. Build the additions of Sec. 5.3: rule wordings (each checked to state the same rules as the original prompt), term lists, prompts without task information.

2. Run E0 and select tasks.

			Task mean			Control mean		
Task	Items	Baseline	L	J	J only	L	J	J only
Noun Class Agreement	100	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Lexical Category Inference	100	0.00	0.00	0.00	0.08	0.00	0.61	0.61
Set Game	10	0.30	−0.09	−0.30	−0.10	−0.30	−0.30	−0.10
pre-answer	10	0.30	0.26	0.29	0.30	0.10	0.00	−0.10
Operator Function	10	0.00	0.00	0.01	0.00	0.00	0.00	0.00
pre-answer	10	0.00	0.00	0.00	0.00	0.00	0.00	0.00
Tapatan	*not yet run*							
pre-answer	*not yet run*							

Table 9: E2 causal check, as Table 4: the rule-style head sets, patched with the means of the seven rule wordings into the prompts without task information; accuracy is averaged over the seven means before the maximum is taken. Control mean: the control task's mean under its original rules prompt. Reasoning-mode tasks: 10 patched items and $\alpha \in \{0, 1, 4, 10\}$, with heads and means taken at the last prompt token or (pre-answer) at the pre-answer position.

			Task mean		
Task	Items	Baseline	L	J	J only
Noun Class Agreement	100	0.00	0.58	0.42	0.42
Lexical Category Inference	100	0.00	0.59	0.33	0.42
Set Game	10	0.00	0.10	0.00	0.03
pre-answer	10	0.00	0.34	0.26	0.21
Operator Function	7	0.00	1.00	0.14	0.29
pre-answer	7	0.00	1.00	0.76	0.20
Tapatan	*not yet run*				
pre-answer	*not yet run*				

Table 10: E2, second causal check, as Table 9, with the original rules prompts of the first 100 items answered wrongly under them patched (10 in reasoning mode; 7 on Operator Function, all it answers wrongly). Unpatched accuracy is 0 by construction. No control mean. Answering every patched item with its more frequent label scores 0.58 on Noun Class Agreement and 0.73 on Lexical Category Inference ("Yes" for both).

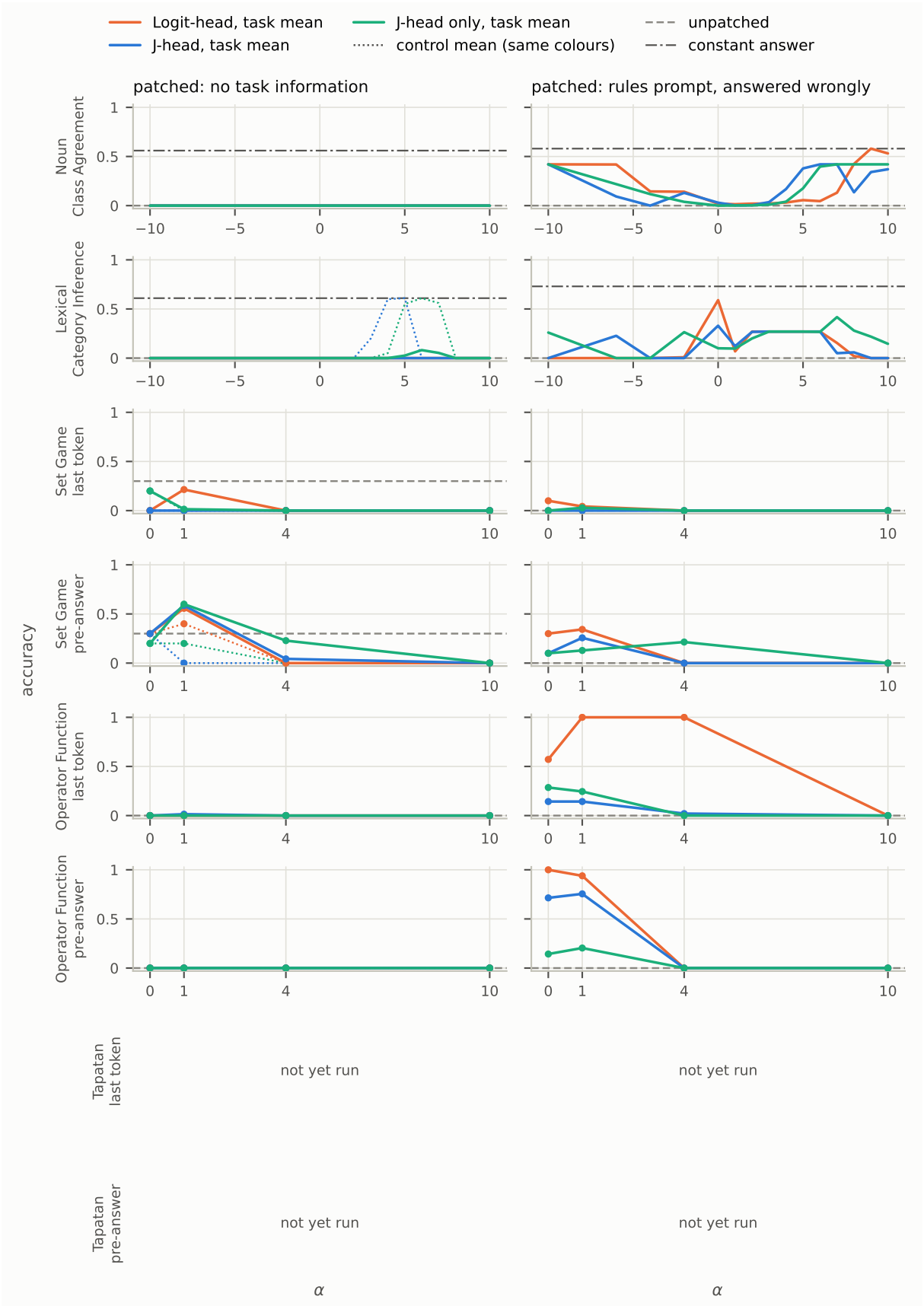


Figure 5: E2 causal checks, as Figure 3: the rule-style head sets, with accuracy averaged over the means of the seven rule wordings. Left: the prompts without task information patched (first check); right: the original rules prompts answered wrongly (second check, no control mean). Reasoning-mode tasks: 10 patched items (7 in Operator Function's second check) and $\alpha \in \{0, 1, 4, 10\}$. Constant answer: the accuracy of answering every patched item with its more frequent label ("No" on the left, "Yes" on the right).

3. Run E1.

4. Run E2.

Reused code and data: the lexical-task-head code of Yang et al. (2026); the Head-JLens code and matrices of Merlo (2026); the data and generation code of Fu et al. (2026); the task datasets of Todd et al. (2024).

Limitations. One model, and one difficulty level of each complex task. The term lists are generated by an LLM and checked by hand, and the identified heads depend on them. Rule-based prompts contain task words, so a head may decode them by copying from the prompt; the copy share measures this but does not decide it. Under J-head, some sites rank tokens highly on unrelated prompts as well (Merlo, 2026, Sec. 3.6), so a larger J-head set is not by itself evidence of more task representation; the causal check is needed. Prompts on which the model explains before answering are left out, which may remove some tasks or styles from E1 and E2.

Deliverables. Head sets, measures and causal-check curves for every task, style and readout.

References

Guy Davidson, Todd M. Gureckis, Brenden M. Lake, and Adina Williams. Do different prompting methods yield a common task representation in language models? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2505.12075.

Xiang Fu, Seungmin Cho, Yukyung Lee, and Najoung Kim. LLMs learn better in-context from rules than from examples. *arXiv preprint arXiv:2609.03213*, 2026.

Wes Gurnee, Nicholas Sofroniew, Adam Pearce, et al. Verbalizable representations form a global workspace in language models. Transformer Circuits Thread, 2026. URL https://transformer-circuits.pub/2026/workspace/index.html.

Filippo Merlo. Reading attention heads through the Jacobian lens. Manuscript, October 2026. https://github.com/filippo-merlo/head_level_jlens, 2026.

nostalgebraist. interpreting GPT: the logit lens. LessWrong, 2020. URL https://www.lesswrong.com/posts/AcKRB8wDpdaN6v6ru/interpreting-gpt-the-logit-lens.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.15213.

Zhuonan Yang, Jacob Xiaochen Li, Francisco Piedrahita Velez, Eric Todd, David Bau, Michael L. Littman, Stephen H. Bach, and Ellie Pavlick. Shared lexical task representations explain behavioral variability in LLMs. In *International Conference on Machine Learning (ICML)*, volume 306 of *PMLR*, 2026. arXiv:2604.22027.

A Original rule prompts

The rules-only prompts of Fu et al. (2026), from their released code (https://github.com/tinlaboratory/rules-vs-examples, `rules-mode-prompts/`, MIT licence). Text in curly braces is filled per item. Tapatan uses the final-board-state prompt, whose levels differ only in its parameters: board size 3, 5, 7 and line length 3, 4, 5 for Easy, Medium, Hard (Fu et al., 2026, Table 1); the version in Fu et al. (2026, App. F) also states the maximum number of pieces

per player and that at most one player has a winning line. The Medium and Hard prompts of Lexical Category Inference are identical within each logic variant.

Set Game

Easy.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has two attributes: an animal and a biome.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is a set of three cards: For each attribute (animal, biome), the three cards
↪  must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Medium.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has three attributes: an animal, a biome, and a food.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. There are 10 types of food: apple, potato, bread, cake, carrot, mutton, beef, cookie,
↪  pie, melon.
5. A GAME-SET is a set of three cards: For each attribute (animal, biome, food), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Hard.

```
You will be shown 9 cards on the board and have to select 3 cards that form a GAME-SET based
↪  on the following rules.

RULES:
1. Each card has a number and three attributes: an animal, a biome, and a food.
2. Numbers range from 1 to 12.
3. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
4. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
5. There are 10 types of food: apple, potato, bread, cake, carrot, mutton, beef, cookie,
↪  pie, melon.
6. A GAME-SET is a set of three cards: For each attribute (animal, biome, food), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value, and 1 of them has a different value, the set is NOT valid.
7. But only for the number, 2 of the cards should have the same number, and 1 of them should
↪  have a different number in order for the set to be valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Tapatan

all levels.

```
You will be learning a new board game. You will then be given one final board state from
↪  that game. Determine the current game outcome.

RULES:
1. The board is a {board_size}x{board_size} grid of points with coordinates (x,y), where 0
↪  <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if they have {line_length} pieces in a straight line (horizontal, vertical,
↪  or diagonal) and each consecutive pair of points in that line is adjacent under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Operator Function

Easy.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.
```

```
RULES:
1. The new operator function f is composed of two arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z be inputs of the function.
3. To compute f(x,y,z), first multiply x and y.
4. Then, add z to the result from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Medium.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function g is composed of three arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z, a be inputs of the function.
3. To compute g(x,y,z,a), first subtract y from x.
4. Second, multiply the result from the previous operation by z.
5. Last, adds a to the result of the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Hard.

```
You will be shown a problem that uses a newly defined operator function based on the
↪  following rules and need to calculate the answer for the problem.

RULES:
1. The new operator function h is composed of four arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z, a, b be inputs of the function.
3. To compute h(x,y,z,a,b), first add x and y.
4. Second, multiply the result from the previous operation by z.
5. Third, subtract a from the result of the previous operation.
6. Last, take the remainder when the result is divided by b.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Noun Class Agreement

Easy.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 2 noun classes: A and B.
2. Each noun phrase has the form DET NOUN.
3. The class A determiner is ka, and the class B determiner is ti.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences follow the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if, in each noun phrase, the determiner matches the
↪  noun's class.
8. Your answer must be exactly one word: Yes or No.
```

```
Sentence: {sentence}
Answer:
```

Medium.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 4 noun classes: A, B, C, D.
2. A noun phrase has the form DET ADJ-SUFF NOUN.
3. The class A determiner is ka, the class B determiner is ti, the class C determiner is su,
↪  and the class D determiner is vo.
4. Adjectives have the form STEM-SUFFIX. The suffix must match the noun's class.
5. The class A adjective suffix is -en, the class B adjective suffix is -os, the class C
↪  adjective suffix is -im, and the class D adjective suffix is -at.
6. Example adjective tokens are glim-en, prun-os, fen-im, zay-at.
7. The class A nouns are tav, fap. The class B nouns are bim, glarn. The class C nouns are
↪  sular, wug. The class D nouns are noko, zesh.
8. Sentences follow the template DET ADJ-SUFF NOUN VERB DET ADJ-SUFF NOUN.
9. Verbs are plain roots like dax, miv, lorp and never change.
10. A sentence is grammatical if and only if, in each noun phrase, both the determiner and
↪  the adjective suffix match the noun's class.
11. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Hard.

```
You will be given one sentence in a synthetic language. Decide whether the sentence is
↪  grammatical.

RULES:
1. There are 6 noun classes: A, B, C, D, E, F.
2. A noun phrase has the form DET ADJ-SUFF NOUN.
3. The class A determiner is ka, the class B determiner is ti, the class C determiner is su,
↪  the class D determiner is vo, the class E determiner is ne, and the class F determiner
↪  is la.
4. Adjectives have the form STEM-SUFFIX. The suffix must match the noun's class.
5. The class A adjective suffix is -en, the class B adjective suffix is -os, the class C
↪  adjective suffix is -im, the class D adjective suffix is -at, the class E adjective
↪  suffix is -uk, and the class F adjective suffix is -esh.
6. The verb has the form PREFIX-ROOT-SUFFIX.
7. The prefix must match the class of the subject noun, and the suffix must match the class
↪  of the object noun.
8. The class A subject prefix is ge-, the class B subject prefix is du-, the class C subject
↪  prefix is ri-, the class D subject prefix is zo-, the class E subject prefix is pa-, and
↪  the class F subject prefix is li-.
9. The class A object suffix is -an, the class B object suffix is -eb, the class C object
↪  suffix is -ig, the class D object suffix is -ot, the class E object suffix is -ul, and
↪  the class F object suffix is -er.
10. Example verb tokens are ge-dax-eb, pa-miv-ig, du-lorp-er.
11. The class A nouns are tav, fap. The class B nouns are bim, glarn. The class C nouns are
↪  sular, wug. The class D nouns are noko, zesh. The class E nouns are tac, fep. The class
↪  F nouns are lurn, prax.
12. Sentences follow the template DET ADJ-SUFF NOUN PREFIX-ROOT-SUFFIX DET ADJ-SUFF NOUN.
13. Verb roots are plain roots like dax, miv, lorp. Only the verb's prefix and suffix change.
14. A sentence is grammatical if and only if the subject noun phrase's determiner and
↪  adjective suffix match the subject noun's class, the object noun phrase's determiner and
↪  adjective suffix match the object noun's class, the verb prefix matches the subject
↪  noun's class, and the verb suffix matches the object noun's class.
```

```
15. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Lexical Category Inference

Easy.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Medium and Hard, OR.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category lists multiple descriptions, an item is a member of that category if it
↪  satisfies at least one listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Medium and Hard, AND.

```
You will be shown a list of four words. Decide whether this list is correct based on the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning shown in the category definitions below.
3. A correct list contains exactly four comma-separated items.
```

```
4. In a correct list, the first item is a member of Category 1, the second item is a member
↪  of Category 2, and so on.
5. If a category lists multiple descriptions, an item is a member of that category only if
↪  it satisfies all listed descriptions.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

B Generation prompts

Both prompts are given to GPT-5.5 through the Azure OpenAI API; `{rule_prompt}` is an original prompt of App. A.

Rule wordings

```
You will be given a prompt that describes the rules of a task for a language model. Write
↪  {N} paraphrases of it.

Requirements:
1. Each paraphrase must state exactly the same rules as the original: the same constraints,
↪  conditions, quantities, names, symbols and labels. Do not add, remove, weaken or
↪  strengthen any rule, and do not add examples, hints or explanations.
2. Change the wording and the sentence structure. Reorder rules only where their order does
↪  not matter; if a rule refers to another rule by its number, keep the reference correct.
3. Copy verbatim every placeholder in curly braces (e.g. {board}), the answer-format
↪  instruction, and every output label or format string (e.g. "Yes or No", "A win, B win,
↪  or continue", \boxed{...}).
4. Keep unchanged the final lines that introduce the input and the answer (e.g. "Sentence:
↪  {sentence}" and "Answer:").
5. The paraphrases must differ from the original and from each other.

Return a JSON list of {N} strings.

Original prompt:
<<<
{rule_prompt}
>>>
```

Term lists

```
You will be given the rules of a task for a language model. List words that describe the
↪  task itself: what the task asks the model to do, its operation or relation, and its key
↪  concepts. For example, for the task "give the antonym of a word": opposite, reverse,
↪  antonym, contrary.

Requirements:
1. Single words only.
2. Do not list possible answers or output labels, words that only name the items of a
↪  particular input (e.g. specific animals, nouns or numbers), or placeholders.
3. Include singular and plural forms and capitalised forms. You may include translations
↪  into other languages.
4. Return a JSON list of strings.

Task rules:
<<<
```

```
{rule_prompt}
>>>
```

C Rule wordings

The six wordings of each original rules prompt of App. A at the easy level, the level in the study, generated and checked as in Sec. 5.3. Text in curly braces is filled per item.

Set Game, easy level

Wording 1.

```
You will see 9 cards on the board, and your task is to choose 3 cards that make a GAME-SET
↪  according to these rules.

RULES:
1. Every card has two attributes: an animal and a biome.
2. The 10 animal types are: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep, turtle,
↪  wolf.
3. The 10 biome types are: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET consists of three cards: For each attribute (animal, biome), the values across
↪  the three cards must be either ALL the SAME or ALL DIFFERENT. For example, if 2 cards
↪  share the same value and 1 card has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 2.

```
You are given a board with 9 cards and must pick 3 cards that constitute a GAME-SET under
↪  the rules below.

RULES:
1. Each card contains two attributes: an animal and a biome.
2. The animals come in 10 types: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. The biomes come in 10 types: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is a set of three cards: For each attribute (animal, biome), the three cards
↪  must have values that are either ALL the SAME or ALL DIFFERENT. For instance, when 2
↪  cards have the same value and 1 card has another value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 3.

```
A board containing 9 cards will be shown, and you need to select 3 cards that form a
↪  GAME-SET by following the rules.

RULES:
1. Each card is defined by two attributes: an animal and a biome.
2. There are 10 animal types: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 biome types: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is made up of three cards: For each attribute (animal, biome), the three cards
↪  must be either ALL the SAME or ALL DIFFERENT. If, for example, 2 of the cards have one
↪  value and 1 has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 4.

```
You will be presented with 9 cards on the board and must identify 3 cards that create a
↪  GAME-SET according to the following rules.

RULES:
1. Each card has two attributes: an animal and a biome.
2. The 10 kinds of animals are: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. The 10 kinds of biomes are: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
4. A GAME-SET is a group of three cards: For each attribute (animal, biome), the three cards
↪  must be either ALL the SAME or ALL DIFFERENT. e.g. when 2 of the cards share the same
↪  value and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 5.

```
From the 9 cards displayed on the board, choose 3 cards that satisfy the definition of a
↪  GAME-SET using these rules.

RULES:
1. Each card includes two attributes: an animal and a biome.
2. There are 10 types of animals: armadillo, axolotl, bat, cat, fox, horse, rabbit, sheep,
↪  turtle, wolf.
3. There are 10 types of biomes: plain, savanna, desert, swamp, forest, jungle, taiga, hill,
↪  badland, tundra.
```

```
4. A GAME-SET is a collection of three cards: For each attribute (animal, biome), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have the same
↪  value while 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Wording 6.

```
You will be shown a board of 9 cards, and you have to determine 3 cards that form a GAME-SET
↪  based on the rules below.

RULES:
1. Each card has two attributes: an animal and a biome.
2. The animal attribute has 10 possible types: armadillo, axolotl, bat, cat, fox, horse,
↪  rabbit, sheep, turtle, wolf.
3. The biome attribute has 10 possible types: plain, savanna, desert, swamp, forest, jungle,
↪  taiga, hill, badland, tundra.
4. A GAME-SET is a set containing three cards: For each attribute (animal, biome), the three
↪  cards must be either ALL the SAME or ALL DIFFERENT. e.g. if 2 of the cards have matching
↪  values and 1 of them has a different value, the set is NOT valid.

Here is the board:

{board}

Based on the rules, tell me which three cards here constitute a GAME-SET. Type the cards in
↪  the following format:
 \boxed{First card: CARD1
Second card: CARD2
Third card: CARD3}
```

Operator Function, easy level

Wording 1.

```
A problem involving a newly defined operator function will be presented, and you must
↪  determine its answer using these rules.

RULES:
1. The new operator function f consists of two arithmetic operators, and all of its inputs
↪  are integers.
2. Let x, y, z be the inputs to the function.
3. When computing f(x,y,z), begin by multiplying x and y.
4. After that, add z to the outcome of the preceding operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 2.

```
You need to solve a problem that uses a newly introduced operator function defined according
↪  to the rules below.
```

```
RULES:
1. The new operator function f is made from two arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z represent the inputs of the function.
3. To evaluate f(x,y,z), multiply x by y first.
4. Next, add z to the result obtained from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 3.

```
A problem will be given that relies on a newly defined operator function; compute the answer
↪  by applying the following rules.

RULES:
1. The new operator function f is formed using two arithmetic operators, and the inputs to
↪  it are integers.
2. Let x, y, z be inputs of the function.
3. For f(x,y,z), first take the product of x and y.
4. Then add z to the result of that prior operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 4.

```
You will receive a problem using a newly defined operator function, and you must calculate
↪  the result according to these rules.

RULES:
1. The new operator function f is built from two arithmetic operators, and its inputs are
↪  integers.
2. Let x, y, z denote the inputs of the function.
3. In computing f(x,y,z), the first step is to multiply x and y.
4. The next step is to add z to the result from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 5.

```
The problem shown will involve a newly defined operator function, and the answer must be
↪  found using the rules that follow.

RULES:
1. The new operator function f is composed of two arithmetic operators, with integer inputs.
2. Let x, y, z be the function's inputs.
3. To calculate f(x,y,z), start by multiplying x and y.
4. Then add z to the value produced by the preceding operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Wording 6.

```
Use the rules below to calculate the answer to a problem involving a newly defined operator
↪  function.

RULES:
```

```
1. The new operator function f contains two arithmetic operators, and the inputs are
↪  integers.
2. Let x, y, z be inputs of the function.
3. To compute f(x,y,z), multiply x and y before anything else.
4. Then add z to the result from the previous operation.
Problem: \({problem}\)
Answer (place the result in \boxed{}):\boxed{}
```

Noun Class Agreement, easy level

Wording 1.

```
You will receive a single sentence in a synthetic language. Determine whether it is
↪  grammatical.

RULES:
1. The language has 2 noun classes: A and B.
2. Every noun phrase is structured as DET NOUN.
3. ka is the determiner for class A, and ti is the determiner for class B.
4. The nouns in class A are tac, sular, fep, wug. The nouns in class B are bim, noko, glarn,
↪  zesh.
5. Sentences use the pattern DET NOUN VERB DET NOUN.
6. The verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical exactly when, in every noun phrase, the determiner matches the
↪  noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 2.

```
You will be provided with one sentence written in a synthetic language. Judge whether the
↪  sentence is grammatical.

RULES:
1. There are 2 noun classes: A and B.
2. A noun phrase always has the form DET NOUN.
3. The determiner for class A is ka, and the determiner for class B is ti.
4. Class A contains the nouns tac, sular, fep, wug. Class B contains the nouns bim, noko,
↪  glarn, zesh.
5. A sentence follows the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if the determiner in each noun phrase matches the
↪  noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 3.

```
You will see one sentence in a synthetic language. Decide if that sentence is grammatical.

RULES:
1. The noun classes are 2 in number: A and B.
2. Each noun phrase is of the form DET NOUN.
3. ka is used as the class A determiner, and ti is used as the class B determiner.
4. tac, sular, fep, wug are the class A nouns. bim, noko, glarn, zesh are the class B nouns.
```

```
5. Sentences have the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if each noun phrase has a determiner that matches
↪  the noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 4.

```
A single sentence in a synthetic language will be given to you. Decide whether it is
↪  grammatical.

RULES:
1. There are 2 noun classes, A and B.
2. Noun phrases take the form DET NOUN.
3. The class A determiner is ka; the class B determiner is ti.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences are built according to DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if, for each noun phrase, the determiner matches
↪  the noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 5.

```
You will get one sentence from a synthetic language. Determine whether the sentence is
↪  grammatical.

RULES:
1. There are 2 noun classes: A and B.
2. The form of every noun phrase is DET NOUN.
3. For class A, the determiner is ka, and for class B, the determiner is ti.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences conform to the template DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
7. A sentence is grammatical if and only if the determiner matches the noun's class in each
↪  noun phrase.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Wording 6.

```
You will be shown one sentence in a synthetic language. Decide whether that sentence is
↪  grammatical.

RULES:
1. The language includes 2 noun classes: A and B.
2. Each noun phrase follows the form DET NOUN.
3. ka is the class A determiner, while ti is the class B determiner.
4. The class A nouns are tac, sular, fep, wug. The class B nouns are bim, noko, glarn, zesh.
5. Sentences follow this template: DET NOUN VERB DET NOUN.
6. Verbs are plain roots like dax, miv, lorp and never change.
```

```
7. A sentence is grammatical if and only if, within every noun phrase, the determiner
↪  matches the noun's class.
8. Your answer must be exactly one word: Yes or No.

Sentence: {sentence}
Answer:
```

Tapatan, easy level

Wording 1.

```
You are going to learn a new board game. After that, you will receive a single final board
↪  state from the game. Decide the present game outcome.

RULES:
1. The board is a {board_size}x{board_size} grid of points whose coordinates are (x,y), with
↪  0 <= x,y < {board_size}.
2. Each board cell holds A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins when they have {line_length} pieces in one straight line (horizontal,
↪  vertical, or diagonal), with every consecutive pair of points in that line adjacent
↪  according to rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 2.

```
You will be taught a new board game, and then you will be shown one final board state from
↪  that game. Identify the game outcome at that moment.

RULES:
1. The board consists of a {board_size}x{board_size} grid of points at coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if they possess {line_length} pieces in a straight line (horizontal,
↪  vertical, or diagonal) and each successive pair of points in that line is adjacent under
↪  rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 3.

```
You will learn the rules for a new board game. You will then be provided with one final
↪  board state from it. Determine what the current outcome of the game is.

RULES:
1. The board is a grid of points with size {board_size}x{board_size} and coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. In a board cell, A denotes player A's piece, B denotes player B's piece, and . denotes an
↪  empty point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins by having {line_length} pieces in a straight line (horizontal, vertical, or
↪  diagonal), provided that each consecutive pair of points in that line is adjacent under
↪  rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 4.

```
You will be introduced to a new board game. Next, one final board state from the game will
↪  be given to you. Work out the current game outcome.

RULES:
1. The board is a {board_size}x{board_size} grid made of points with coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if {line_length} of their pieces form a straight line (horizontal,
↪  vertical, or diagonal) and every consecutive pair of points in that line is adjacent
↪  under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 5.

```
You will receive the rules of a new board game. Then you will be given one final state of
↪  the board from that game. Determine the outcome currently shown.

RULES:
1. The board is a {board_size}x{board_size} point grid using coordinates (x,y), where 0 <=
↪  x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
```

```
4. A win occurs for a player if they have {line_length} pieces in a straight line
↪  (horizontal, vertical, or diagonal) and each consecutive pair of points in that line is
↪  adjacent under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Wording 6.

```
You will study a new board game and then receive one final board state from that game.
↪  Decide which current game outcome applies.

RULES:
1. The board is a {board_size}x{board_size} grid of points, each with coordinates (x,y),
↪  where 0 <= x,y < {board_size}.
2. A board cell contains A for player A's piece, B for player B's piece, or . for an empty
↪  point.
3. Orthogonal neighbors are always adjacent. Diagonal neighbors are adjacent only from
↪  even-parity points where (x+y) is even: (x,y) connects to (xś1,yś1) when in bounds.
4. A player wins if they have {line_length} pieces arranged in a straight line (horizontal,
↪  vertical, or diagonal) and each consecutive pair of points in that line is adjacent
↪  under rule 3.
5. If A has a valid winning line, output A win. If B has a valid winning line, output B win.
↪  If neither player has a valid winning line, output continue.
6. You must output a label that is exactly one of: A win, B win, or continue.

BOARD:
{board_state}

LABEL:
```

Lexical Category Inference, easy level

Wording 1.

```
You will receive a list made up of four words. Determine whether the list is correct
↪  according to these rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. The meaning of each category is given in the category definitions below.
3. A correct list has exactly four items separated by commas.
4. For a list to be correct, item one must belong to Category 1, item two must belong to
↪  Category 2, and this pattern continues in order.
5. When a category says "at least one", an item may match any of the listed descriptions.
↪  When it says "all", an item must match every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 2.

```
A list of four words will be presented to you. Judge whether it is correct using the rules
↪  below.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category means what is specified in the category definitions below.
3. A correct list consists of exactly four comma-separated items.
4. In a correct list, the items correspond in order: the first item is in Category 1, the
↪  second item is in Category 2, and so on.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 3.

```
You will be given four words as a list. Decide if that list is correct by applying the
↪  following rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. The category definitions below state the meaning of each category.
3. To be correct, a list must include exactly four items separated by commas.
4. To be correct, the first item must be a member of Category 1, the second item must be a
↪  member of Category 2, and so forth.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 4.

```
You will see a four-word list. Assess whether this list is correct under the rules that
↪  follow.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category has the meaning provided in the category definitions below.
3. A list is correct only if it contains exactly four comma-separated items.
4. A correct list places a Category 1 member first, a Category 2 member second, and
↪  continues in that order.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}
```

```
List: {candidate_block}
Answer:
```

Wording 5.

```
You will be shown four words in list form. Decide whether the list qualifies as correct
↪  according to these rules.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. The definitions below give the meaning for each category.
3. Exactly four comma-separated items are required for a correct list.
4. In a correct list, the first, second, and later items must respectively be members of
↪  Category 1, Category 2, and the following categories in sequence.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

Wording 6.

```
A list containing four words will be provided. Determine whether it is correct based on the
↪  rules listed here.

RULES:
1. There are exactly four categories: {prompt_labels}.
2. Each category's meaning is the one shown in the category definitions below.
3. For a list to be correct, it must contain exactly four items, with commas separating them.
4. For a list to be correct, its first item must belong to Category 1, its second item must
↪  belong to Category 2, and the remaining items must follow the same order.
5. If a category says "at least one", an item may satisfy any listed description. If it says
↪  "all", an item must satisfy every listed description.
6. Your answer must be exactly one word: Yes or No.

CATEGORY DEFINITIONS:
{label_block}

List: {candidate_block}
Answer:
```

D Term lists

The term list of each complex task at the easy level, generated and checked as in Sec. 5.3, as words before their expansion to token ids. Tapatan's list is made from its single prompt, shared by all levels.

Task (easy level)	Terms
Set Game	set, sets, Set, Sets, card, cards, Card, Cards, board, boards, Board, Boards, select, selects, Select, Selects, identify, identifies, Identify, Identifies, find, finds, Find, Finds, choose, chooses, Choose, Chooses, match, matches, Match, Matches, matching, Matching, valid, Valid, invalid, Invalid, attribute, attributes, Attribute, Attributes, animal, animals, Animal, Animals, biome, biomes, Biome, Biomes, same, Same, different, Different, all, All, triple, triples, Triple, Triples, triad, triads, Triad, Triads, combination, combinations, Combination, Combinations, group, groups, Group, Groups, game, games, Game, Games, gameset, gamesets, Gameset, Gamesets, rule, rules, Rule, Rules, constraint, constraints, Constraint, Constraints, value, values, Value, Values
Operator Function	arithmetic, Arithmetic, operator, operators, Operator, Operators, function, functions, Function, Functions, calculation, calculations, Calculation, Calculations, compute, Compute, calculate, Calculate, evaluate, Evaluate, solve, Solve, multiplication, multiplications, Multiplication, Multiplications, multiply, Multiply, product, products, Product, Products, addition, additions, Addition, Additions, add, Add, sum, sums, Sum, Sums, integer, integers, Integer, Integers, input, inputs, Input, Inputs, result, results, Result, Results, expression, expressions, Expression, Expressions, composition, compositions, Composition, Compositions
Noun Class Agreement	grammar, Grammar, grammars, Grammars, grammaticality, Grammaticality, grammatical, Grammatical, ungrammatical, Ungrammatical, syntax, Syntax, sentence, Sentence, sentences, Sentences, phrase, Phrase, phrases, Phrases, noun, Noun, nouns, Nouns, determiner, Determiner, determiners, Determiners, class, Class, classes, Classes, agreement, Agreement, match, Match, matches, Matches, matching, Matching, mismatch, Mismatch, mismatches, Mismatches, template, Template, templates, Templates, validation, Validation, classification, Classification, decision, Decision, decisions, Decisions, binary, Binary, well-formedness, Wellformedness, acceptability, Acceptability
Tapatan	board, boards, Board, Boards, game, games, Game, Games, grid, grids, Grid, Grids, cell, cells, Cell, Cells, point, points, Point, Points, coordinate, coordinates, Coordinate, Coordinates, piece, pieces, Piece, Pieces, player, players, Player, Players, state, states, State, States, outcome, outcomes, Outcome, Outcomes, result, results, Result, Results, winner, winners, Winner, Winners, win, wins, Win, Wins, winning, Winning, line, lines, Line, Lines, alignment, alignments, Alignment, Alignments, sequence, sequences, Sequence, Sequences, straight, Straight, horizontal, Horizontal, vertical, Vertical, diagonal, diagonals, Diagonal, Diagonals, orthogonal, Orthogonal, neighbor, neighbors, Neighbor, Neighbors, adjacency, adjacencies, Adjacency, Adjacencies, adjacent, Adjacent, connection, connections, Connection, Connections, connectivity, Connectivity, parity, parities, Parity, Parities, even, Even, empty, Empty, validity, Validity, valid, Valid, classification, classifications, Classification, Classifications, classify, Classify, determine, Determine, detect, Detect, evaluate, Evaluate, identify, Identify, label, labels, Label, Labels
Lexical Category Inference	validation, Validation, validate, Validate, verification, Verification, assessment, Assessment, decision, Decision, classification, Classification, classify, Classify, categorization, Categorization, categorize, Categorize, category, Category, categories, Categories, membership, Membership, member, Member, members, Members, list, List, lists, Lists, item, Item, items, Items, word, Word, words, Words, order, Order, ordering, Ordering, sequence, Sequence, position, Position, positions, Positions, alignment, Alignment, matching, Matching, match, Match, criteria, Criteria, criterion, Criterion, rule, Rule, rules, Rules, constraint, Constraint, constraints, Constraints, definition, Definition, definitions, Definitions, correctness, Correctness, incorrectness, Incorrectness, satisfaction, Satisfaction, validity, Validity

Decision log

2026-10-27 Scope: test the lexical-task-head method of Yang et al. (2026) on complex tasks and compare the per-head logit lens with Head-JLens. New title.

2026-10-27 Tasks: the 14 simple tasks shared by Yang et al. (2026) and Davidson et al. (2025); the five tasks of Fu et al. (2026).

2026-10-27 No main condition: rule-based and example-based prompts are treated alike (replacing the earlier choice of rules as main and examples as secondary).

2026-10-27 Method: lexical task heads, with Logit-head and J-head readouts. Function vectors, function-specific heads and the hidden-state difference between prompts are not used for now.

2026-10-27 At least six rule wordings per complex task and difficulty level.

2026-10-27 Term lists are task-specific, not item-specific; one per task, or per difficulty level where the rules change.

2026-10-27 Answer format as in the source studies.

2026-10-27 Model: Llama-3.1-8B-Instruct, the main model of Yang et al. (2026) and the only one with released Head-JLens matrices.

2026-10-27 Rule wordings and term lists for the complex tasks are generated with GPT-5.5 through ChatGPT and checked by hand.

2026-10-27 Accuracy is computed with each source paper's evaluation.

2026-10-27 Prompts on which the model does not answer straight away are left out; other ways of handling them are considered after E0.

2026-10-27 After an early E0 finding (no direct answers on Set Game and Operator Function), these tasks are run in reasoning mode (Sec. 5.3).

2026-10-27 Tapatan is run in reasoning mode as well: in E0 none of its 300 items is answered straight away, under any prompt.

2026-10-27 Complex tasks: the easy level of each task only.

2026-10-27 Rule wordings and term lists are generated with GPT-5.5 through the Azure OpenAI API, replacing ChatGPT; six wordings per original rule prompt. Both were checked by hand and approved.

2026-10-27 Complex tasks are decoded greedily, and use 300 test items per task drawn with the sampler and seed of Fu et al. (2026) (all 600 for Lexical Category Inference), so that the causal check can leave the patched items out of the mean.

2026-10-27 Reasoning mode: 100 items per task, style and wording; the causal check patches 10 items with $\alpha \in \{0, 1, 4, 10\}$.

2026-10-28 The simple tasks follow the code of Yang et al. (2026) exactly, except for the J-head readout and the bfloat16 precision of the Head-JLens matrices: five instruction templates (their code and paper do not use the sixth template of their instruction file); heads from one wording per style (5-shot, and the most accurate template); one mean and one patch per source wording; their control mean; their batching. This replaces, for the simple tasks, the pooling of prompts and means over wordings and the use of six templates. E1 was rerun.

2026-10-28 Simple tasks: the padding of each batch is masked. The code of Yang et al. (2026) passes no attention mask, so the model attends to the padding, which their paper does not describe. E1 was rerun.

2026-10-28 Simple tasks: a second causal check with the setting of the one causal script of Yang et al. (2026) (wrongly answered 1-shot prompts patched, no control). It reproduces their Figs. 57 and 62, which the zero-shot procedure of their App. G.1 does not; both checks are reported.

2026-10-28 Complex tasks follow the simple-task rules: rule-style heads from the most accurate rule wording (replacing the pooling over the seven wordings), and in the causal check one mean and one patch per rule wording; the control mean is the control task's mean under its original rules prompt. The causal check of the complex tasks patches the rule-style head sets only. The second check is adapted to them (original rules prompts answered wrongly, no control). Reasoning mode uses 300 items, as the other complex tasks (replacing 100); the first 100 are the earlier sample, whose replies are reused. E2 was rerun.

2026-10-29 Reasoning mode: each distinct prompt is answered once, and its reply scored for every item that shares it (Tapatan's 300 items repeat 46 boards, and a repeated prompt could get a different greedy reply in another batch).